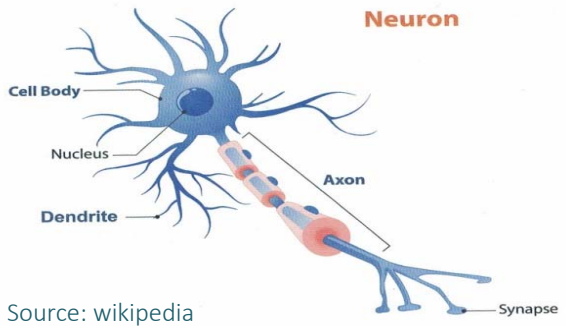
Chapter 6 Neural Networks & Deep Learning

- ❑ Artificial Neural Networks
- ❑ Multilayer Perceptron-Backpropagation
- ❑ Deep Learning

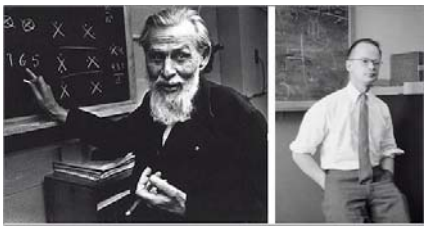
Asst. Prof.Rapeeporn Chamchong, PhD
Department of Computer Science, Faculty of Informatics, MSU

Artificial neuron

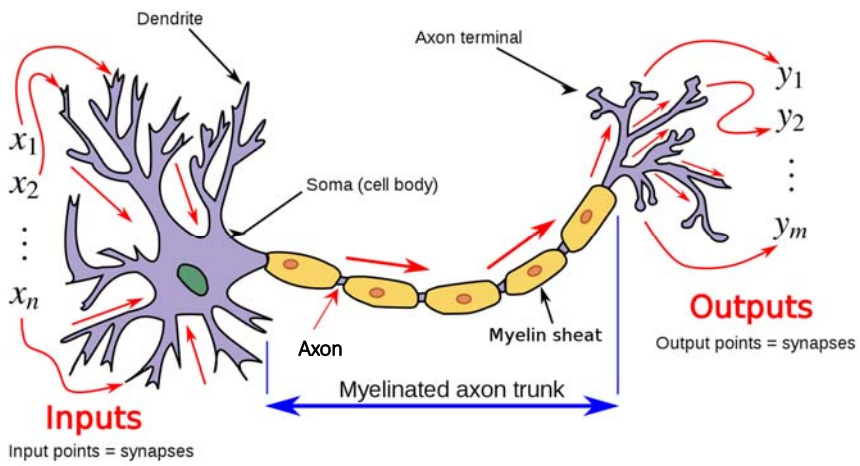
- An artificial neuron is a mathematical function conceived as a model of biological neurons, a neural network.
- Artificial neurons are elementary units in an Artificial Neural Network (ANN).
- Threshold Logic Unit (TLU)
  - The early artificial neuron was the TLU, or Linear Threshold Unit, first proposed by Warren McCulloch and Walter Pitts in 1943.

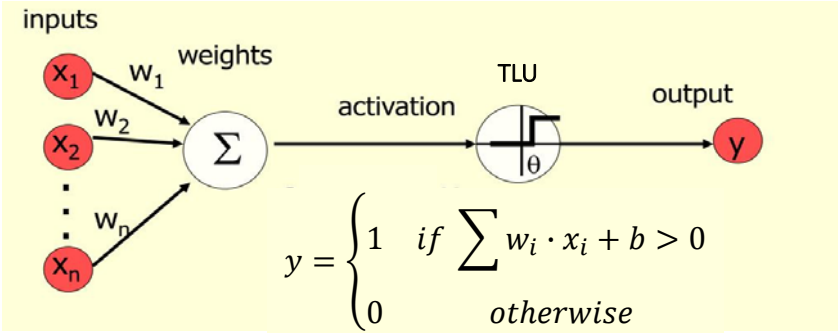


Source: wikipedia



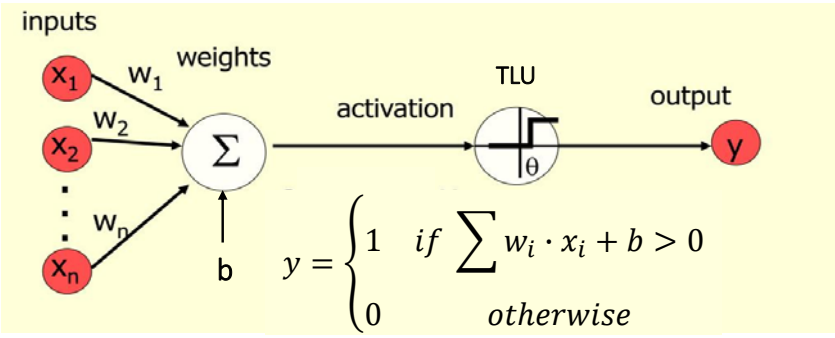
Neuron Network & Mathematical Model



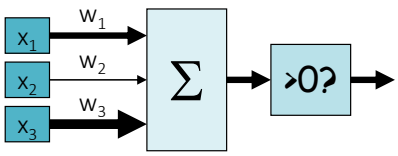


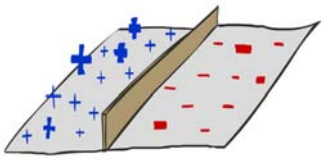
Structure and Properties

- Input nodes are feature values
- Each feature has weights & bias
- Sum is the activation



- Activation function with threshold:
  - Positive, output +1
  - Negative, output -1





Artificial Neural Networks (ANNs)

- 1940: Mcculloch and Pitts
  - A logical calculus of the ideas immanent in nervours activity
- 1958: Frank Rosenblatt
  - The perceptron learning algorithm, convergence theorem
- 1960: Minsky and Papert
  - Perceptrons cannot deal with XOR
- 1980: Werbos and Rumelhart
  - Back-propagation learning algorithm
- 1980: Hopfield
  - Hopfield's energy approach (recurrent neural network)
- 1980 - now: Fukushima, LeCun, Hinton, Schmidhuber, etc.

Types of ANNs

- Feed-Forward NN
  - Single-layer perceptron
  - Multi-layer perceptron
  - Radial Basis Function
- Recurrent NN
  - Competitive Networks
  - Kohonen's Self-organizing Maps (KSOM)
  - Hopefield Network
  - Adaptive Resonance Theory (ART) models

Single Layer vs. Multi-Layer Perceptron

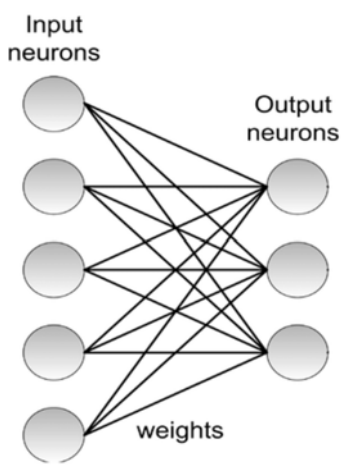
SINGLE LAYER PERCEPTRON

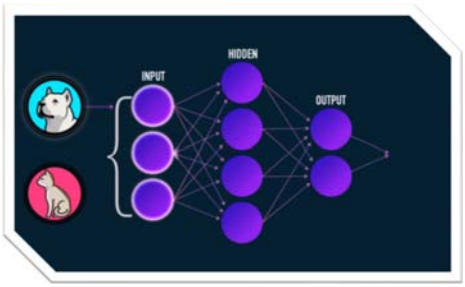
- Perform pattern classification only on linearly separable e.g. AND gate, OR Gate

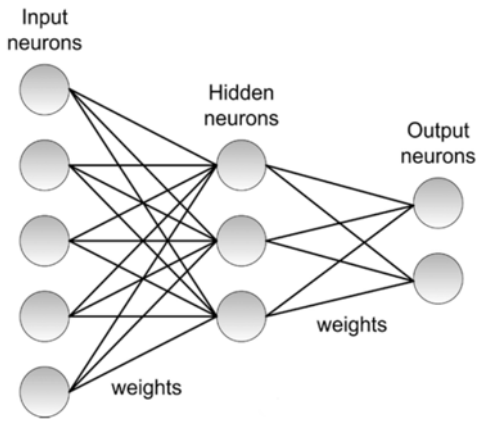
MULTI-LAYER PERCEPTRON (MLP)

- Use for complex problems (require multiple decision lines for a good acceptable solution e.g. XOR gate).
- Have one or more hidden layers.
- Each neuron in the hidden layer is responsible for a different line and they form a classification for the given problem.

Single Layer vs. Multi-Layer Perceptron

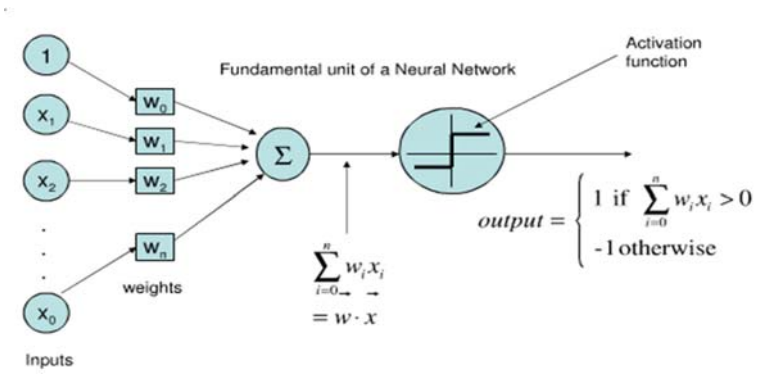






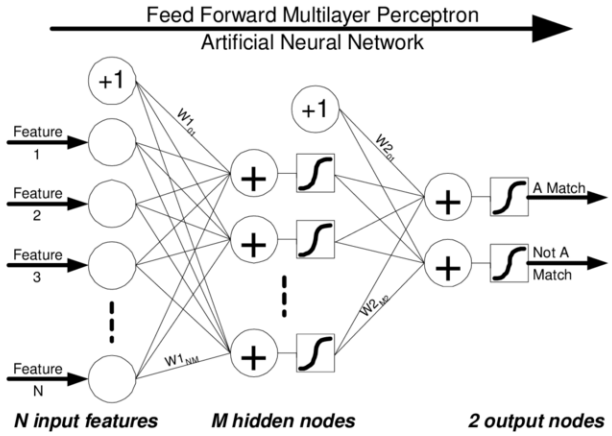
Single Layer vs. Multi-Layer Perceptron

SINGLE LAYER PERCEPTRON



https://www.researchgate.net/figure/Single-Layer-Perceptron-Network-Multilayer-Perceptron-Network-These-type-of-feed-forward_fig2_283339701

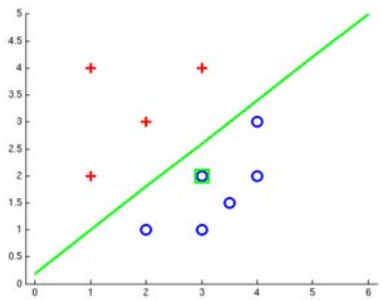
MULTI-LAYER PERCEPTRON



https://www.researchgate.net/figure/The-feed-forward-multilayer-perceptron-artificial-neural-network-architecture-used-to_fig1_221226040

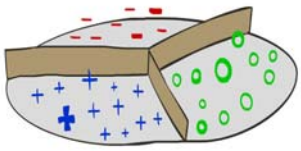
Examples: Perceptron

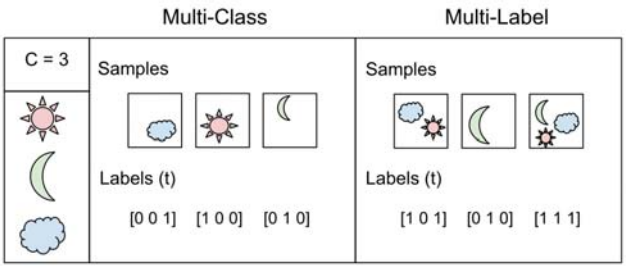
- Binary class decision



Example-playground of ANN:
https://playground.tensorflow.org/

- Multiclass decision





Understanding Categorical Cross-Entropy Loss, Binary Cross-Entropy Loss, Softmax Loss, Logistic Loss, Focal Loss and all those confusing names (gombru.github.io)

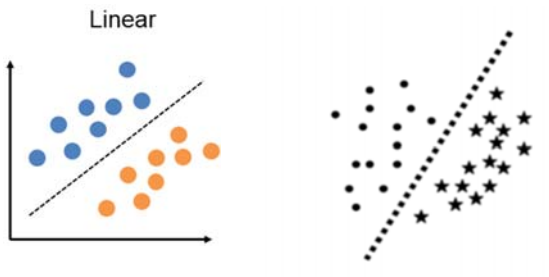
Linearly Separable vs. Non Linearly Separable

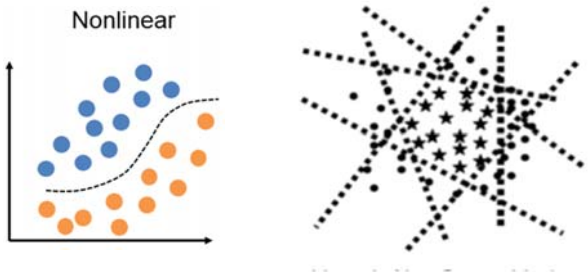
LINEARLY SEPARABLE PROBLEMS

- Represents those problems whose input can be classified using a straight line.
- The simplest examples are the logical AND or OR gate.

NON LINEARLY SEPARABLE PROBLEMS

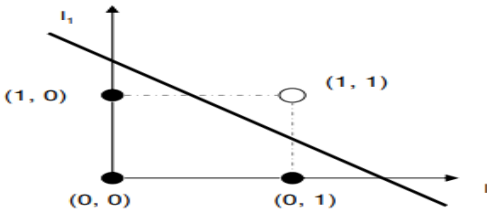
- The data for these problems cannot be separated into their respective classes by using a single straight line



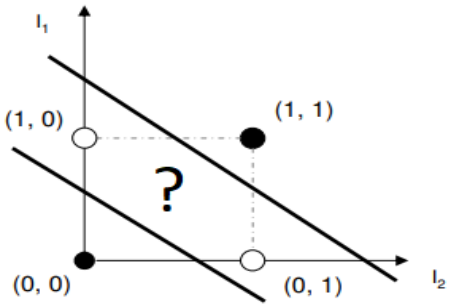


Example: Linearly & Non linearly separable problems

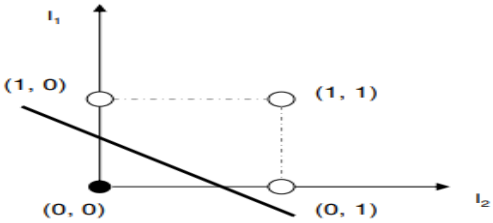
AND		
I_1	I_2	out
0	0	0
0	1	0
1	0	0
1	1	1

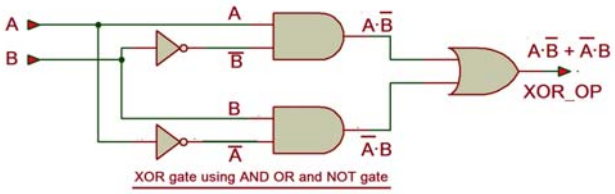


XOR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	0



OR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	1





Linearly separable problems

Non linearly separable problems

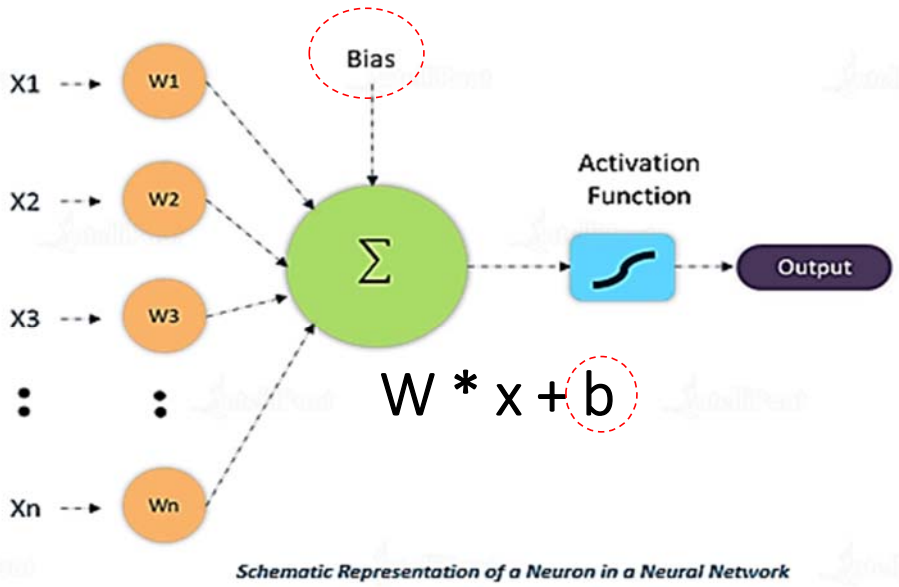
How to determine the values of W?

- How to solve for W

$$f(x) = \begin{cases} 1 & if \sum w_i \cdot x_i + b > 0 \\ 0 & otherwise \end{cases}$$

  - Randomly
  - Analytically
  - Optimization algorithm: e.g. gradient descent

What is Bias



Learning Approaches

- Hebbian rules: cells that fire together wire together
- Competitive learning: increasing specialization of cells
- Hopfield networks: associative memory
- Error correction: backpropagation that can reduce the cost function

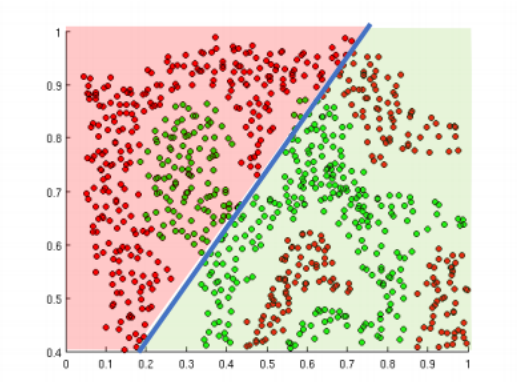
output = sum (weights * inputs) + bias

- Output is calculated and passing it through an activation function like the Sigmoid function, etc.
  - Bias acts like a constant which helps the model to fit the given data.
  - Sigmoid depends on the weight of the inputs.
- Compare bias through a constant c of a linear function

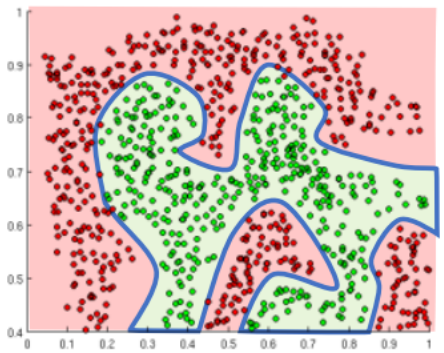
$$y = mx + c$$

Importance of Activation Functions

The purpose of activation functions is to ***introduce non-linearities*** *into the network*



Linear Activation functions produce linear decisions no matter the network size



Non-linearities allow us to approximate arbitrarily complex functions

Activation Function (Transfer Function)

- Binary step activation function:

$$f(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

- Sigmoid activation function (logistic):

$$f(x) = \frac{1}{1 + e^{-x}}, \qquad 0 \leq f(x) \leq 1$$

  - Having vanishing gradient problem

- TanH activation function (Hyperbolic tangent) :

$$tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}, -1 \leq \tanh(x) \leq 1$$

  - Similar to Sigmoid but wider range

- ReLU activation function (Rectified Linear Unit):

$$f(x) = \begin{cases} 0, & x < 0 \\ x, & otherwise \end{cases}$$

  - Using less memory & processing time

- Leaky ReLU activation function :
  - Few change of negative output, the slope will decrease if the input is negative value

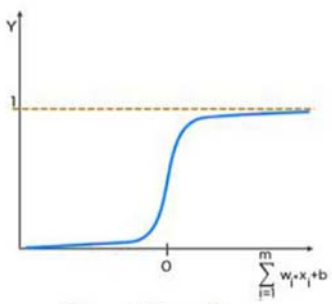
- Softmax function:

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}, 0 \leq \sigma(\vec{z})_i \leq 1$$

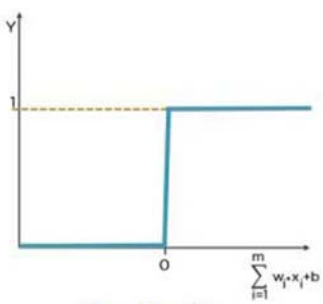
  - Result is a probability. Using in the final output of NN

σ = Softmax function, $\vec{z}$ =input vector, K = number of class, e^{z_i}, e^{z_j} = standard exponential function for input & output vector

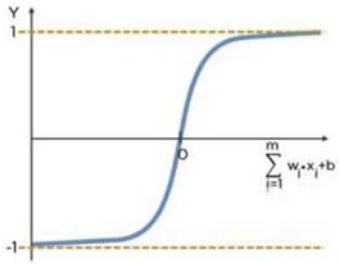
Activation Function (Transfer Function)



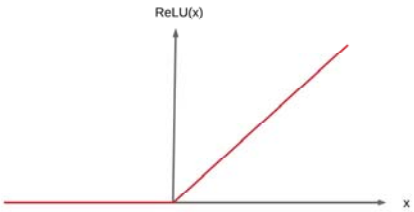
Sigmoid function

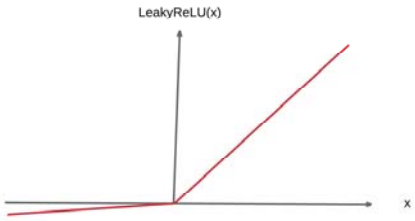


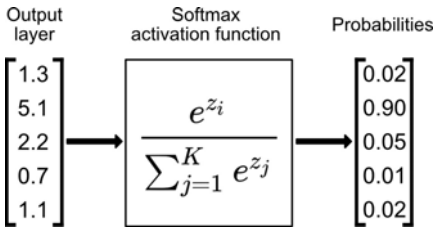
Step function



Tanh function







Adaptive Learning Rate Algorithms

- Momentum — `tf.train.MomentumOptimizer`
- Adagrad — `tf.train.AdagradOptimizer`
- Adadelta — `tf.train.AdadeltaOptimizer`
- Adam — `tf.train.AdamOptimizer`
- RMSProp — `tf.train.RMSPropOptimizer`

Qian et al. "On the momentum term in gradient descent learning algorithms." 1999.

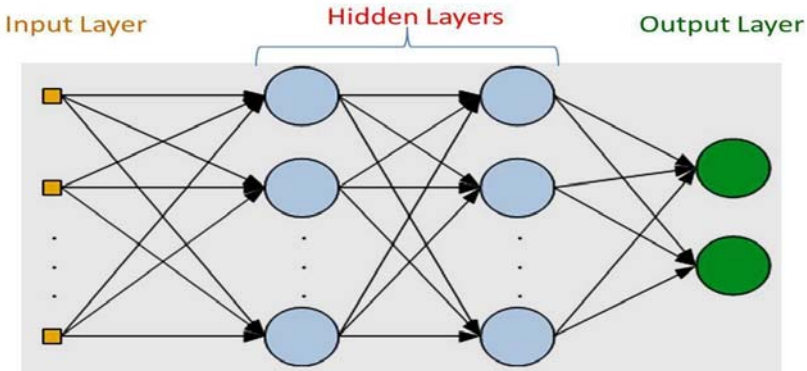
Duchi et al. "Adaptive Subgradient Methods for Online Learning and Stochastic Optimization." 2011.

Zeiler et al. "ADADELTA: An Adaptive Learning Rate Method." 2012.

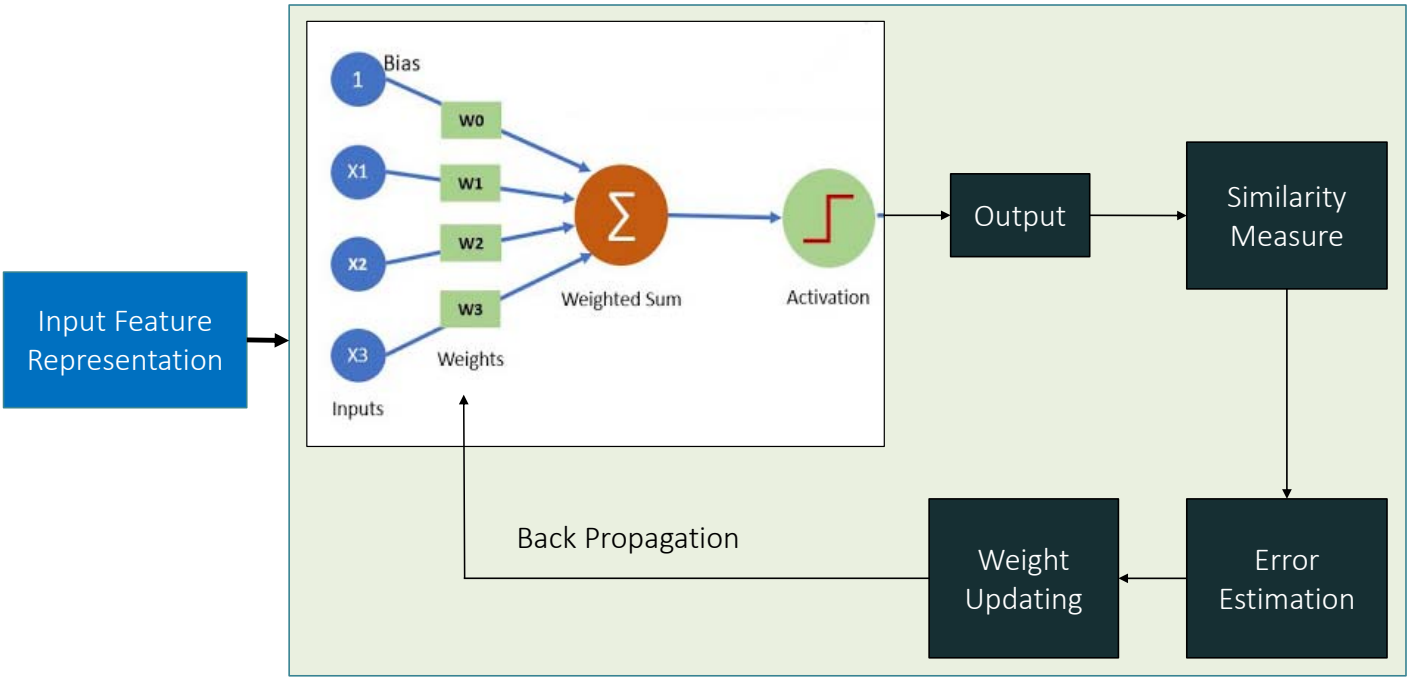
Kingma et al. "Adam: A Method for Stochastic Optimization." 2014.

Multi-layer Perceptron

- Using XOR gate to predict the output
- Three main layers: input, hidden, and output layers
- Hidden layers may be more than one
- This is the main concept of deep learning



Back propagation algorithm



Example: Classification using ANN

X_1	X_2	Y
1.0	1.0	1
9.4	6.4	-1
2.5	2.1	1
8.0	7.7	-1
0.5	2.2	1
7.9	8.4	-1
7.0	7.0	-1
2.8	0.8	1
1.2	3.0	1
7.8	6.1	-1

Training set

Example: Classification using ANN

Prediction (y`) = 1 if Wx+b > 0 and -1 if Wx+b ≤ 0

- Let [0.75, 0.5, –0.6] be the initial weights (set randomly)
- Let the learning rate be 0.2
- The first data sample is classified correctly

$$X_1 * W_1 + X_2 * W_2 + Bias * W3$$
$$= 1 * 0.75 + 1 * 0.5 + 1 * (-0.6)$$
$$= 0.65 >= 0 \rightarrow 1$$

- Perform Same step with second training Example

$$X_1 * W_1 + X_2 * W_2 + Bias * W3$$
$$= 9.4 * 0.75 + 6.4 * 0.5 + 1 * (-0.6)$$
$$= 9.65 >= 0 \quad ? \; 1$$

- (Wrong Classification)

- $\boldsymbol{Error = Desired\ Output - Actual\ Output}$

$$Error = -1 - 1 = -2$$

- $\boldsymbol{W_{new} = W_{old} + 0.2 * Error * X}$

$$W_{new} = \begin{bmatrix} -3.01 \\ -2.06 \\ -1.00 \end{bmatrix} = \begin{bmatrix} 0.75 \\ 0.5 \\ -0.6 \end{bmatrix} + 0.2\,(-2) \begin{bmatrix} 9.4 \\ 6.4 \\ 1 \end{bmatrix}$$

Assuming, after 500 iterations the weights converge to [-1.3, -1.1, 10.9]
So the decision boundary = $-1.3x_1 - 1.1x_2 + 10.9$

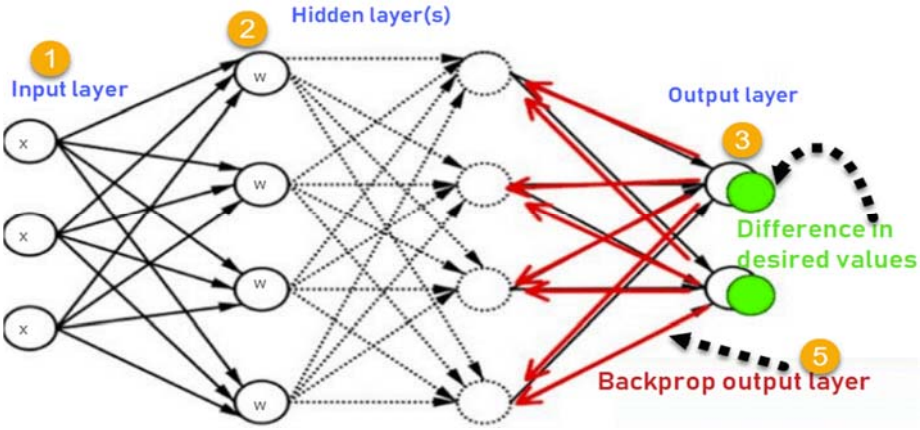
Testing

- Decision Boundary **= $-1.3x_1 - 1.1x_2 + 10.9$**
- **For new test record (x1, x2) (5.6,1.7)**
  =(-1.3)(5.6)+(-1.1)(1.7)+(1)(10.9)
  =-7.28-1.87+10.9
  =1.75 $>= 0 \rightarrow 1$

Predicted Label = 1

Prediction (y`) = 1 if Wx+b > 0 and -1 if Wx+b ≤ 0

MLP – Backpropagation



Mean square error loss

$$E = (t - y)^2$$

What is Gradient Descent?

- An optimization technique is used to improve NN & DL models by minimizing the cost function (loss function).
- A process occurs in the backpropagation phase where the goal is to continuously resample the gradient of the model's parameter in the opposite direction based on the weight w, updating consistently until we reach the global **minimum of function** J(w).
- More Precisely,
  - An algorithm is used to iterate through different combinations of weights in an optimal way.....to find the best combination of weights that has a minimum error.

Training Neural Networks

- Loss Optimization
  - We want to find the network weights that achieve the lowest loss

$$\boldsymbol{\theta}^* = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}\left(f\left(x^{(i)}; \boldsymbol{\theta}\right), y^{(i)}\right)$$

$$\boldsymbol{\theta}^* = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} J(\boldsymbol{\theta})$$

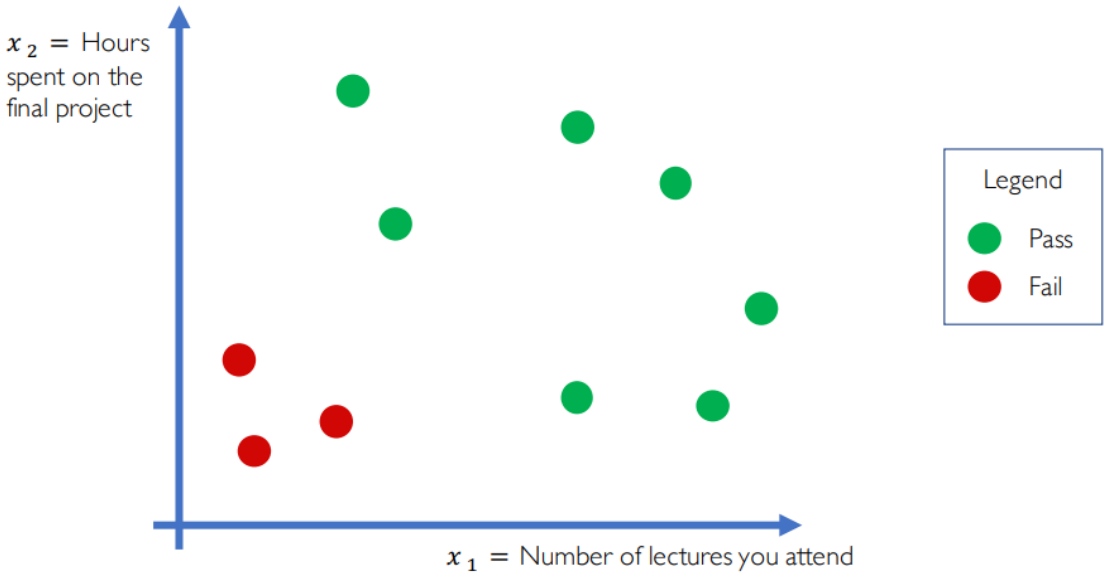
Remember:

Optimization through gradient descent

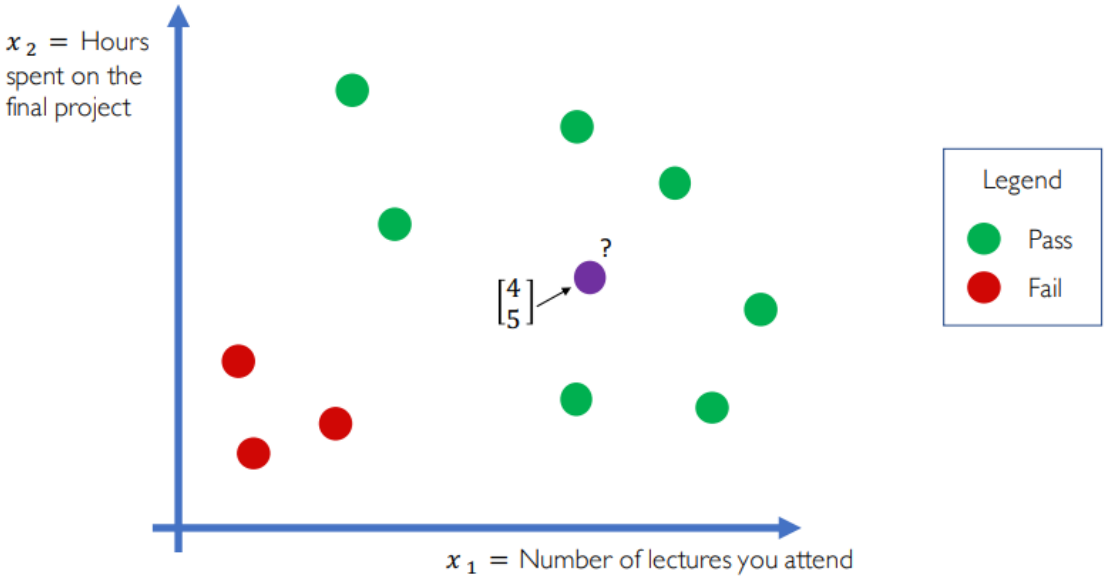
$$\theta \leftarrow \theta - \eta \frac{\partial J(\theta)}{\partial \theta}$$

How can we set the learning rate?

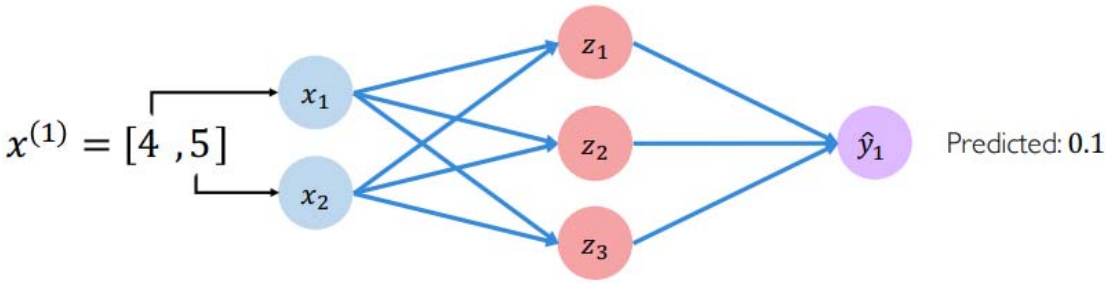
Importance of Activation Functions



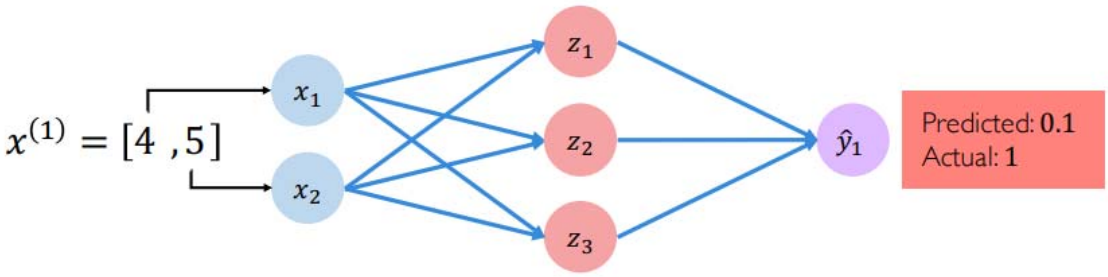
Importance of Activation Functions



Example Problem: Will I pass this course?

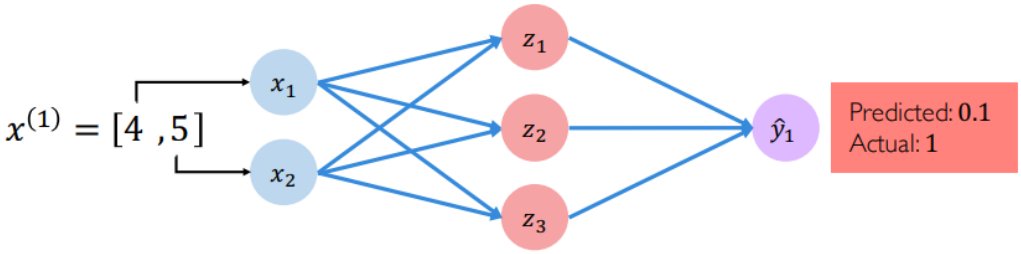


Example Problem: Will I pass this course?



Quantifying Loss

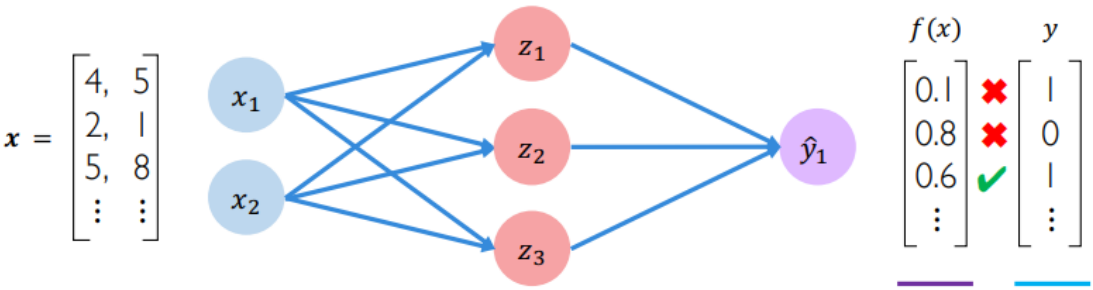
*The **loss** of our network measures the cost incurred from incorrect predictions*



$$\mathcal{L}\big(\underbrace{f\left(x^{(i)};\boldsymbol{\theta}\right)}_{\text{Predicted}}, \underbrace{y^{(i)}}_{\text{Actual}}\big)$$

Empirical Loss

*The **empirical loss** measures the total loss over our entire dataset*



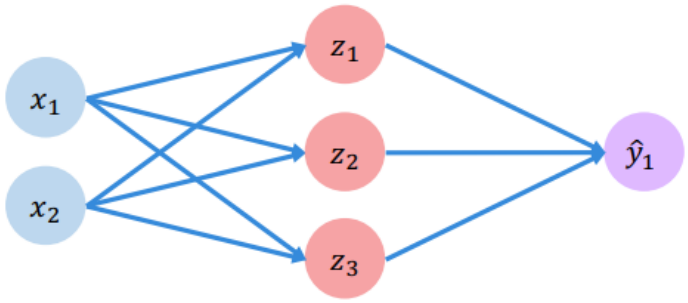
Also known as:
- Objective function
- Cost function
- Empirical Risk

$$\boldsymbol{J}(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n} \mathcal{L}\big(\underbrace{f\left(x^{(i)};\theta\right)}_{\text{Predicted}}, \underbrace{y^{(i)}}_{\text{Actual}}\big)$$

Binary Cross Entropy Loss

Cross entropy loss *can be used with models that output a probability between 0 and 1*

$$\boldsymbol{x} = \begin{bmatrix} 4, & 5 \\ 2, & 1 \\ 5, & 8 \\ \vdots & \vdots \end{bmatrix}$$



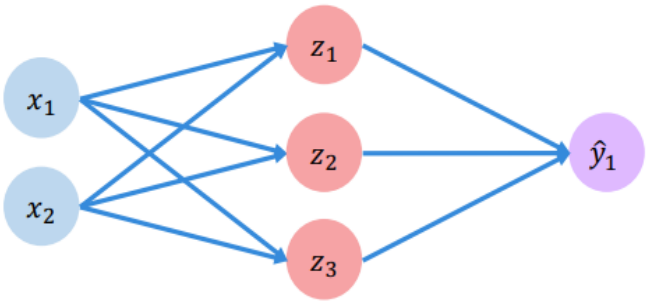
$$\begin{array}{cc} f(x) & y \\ \begin{bmatrix} 0.1 \\ 0.8 \\ 0.6 \\ \vdots \end{bmatrix} \begin{matrix} \times \\ \times \\ \checkmark \\ \\ \end{matrix} & \begin{bmatrix} 1 \\ 0 \\ 1 \\ \vdots \end{bmatrix} \end{array}$$

$$\boldsymbol{J}(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n} \underbrace{y^{(i)}}_{\text{Actual}} \log\left(\underbrace{f\left(x^{(i)};\theta\right)}_{\text{Predicted}}\right) + (1 - \underbrace{y^{(i)}}_{\text{Actual}}) \log\left(1 - \underbrace{f\left(x^{(i)};\theta\right)}_{\text{Predicted}}\right)$$

Mean Squared Error Loss

Mean squared error loss *can be used with regression models that output continuous real numbers*

$$\boldsymbol{x} = \begin{bmatrix} 4, & 5 \\ 2, & 1 \\ 5, & 8 \\ \vdots & \vdots \end{bmatrix}$$

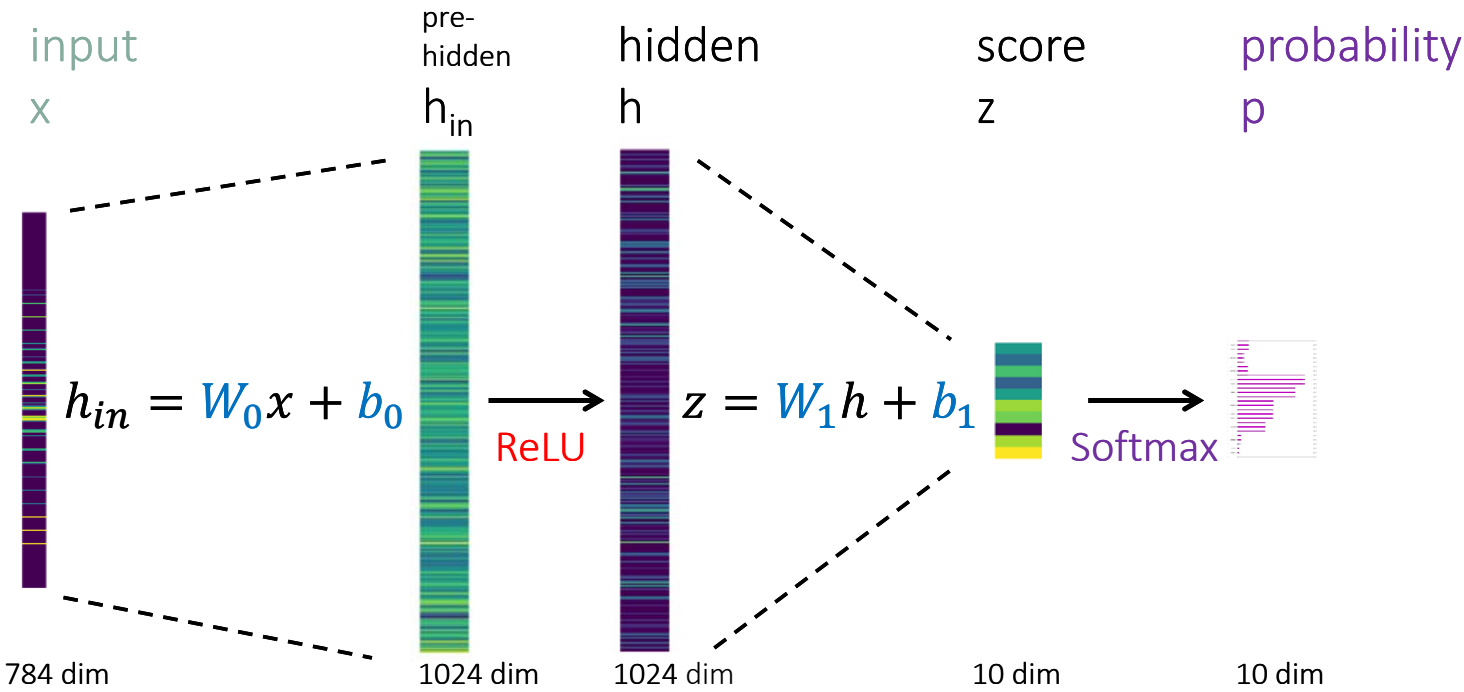


$$\underbrace{\begin{array}{cc} f(x) & y \\ \begin{bmatrix} 30 \\ 80 \\ 85 \\ \vdots \end{bmatrix} \begin{matrix} \times \\ \times \\ \checkmark \\ \\ \end{matrix} & \begin{bmatrix} 90 \\ 20 \\ 95 \\ \vdots \end{bmatrix} \end{array}}_{\text{Final Grades (percentage)}}$$

$$\boldsymbol{J}(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n} \left(\underbrace{y^{(i)}}_{\text{Actual}} - \underbrace{f\left(x^{(i)};\theta\right)}_{\text{Predicted}}\right)^2$$

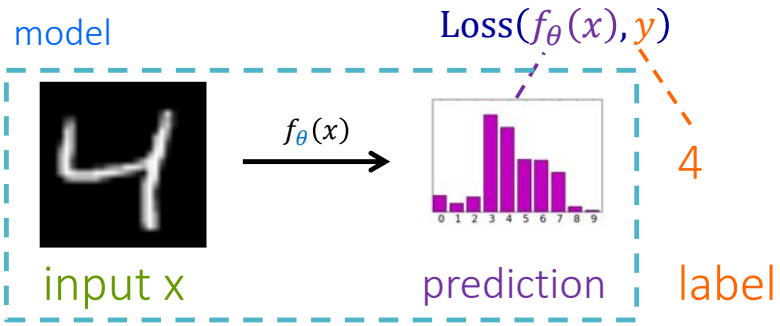
Going deeper

W_0, b_0, W_1, b_1: parameters



Learning objective

- Objective: minimize



θ: parameters

-

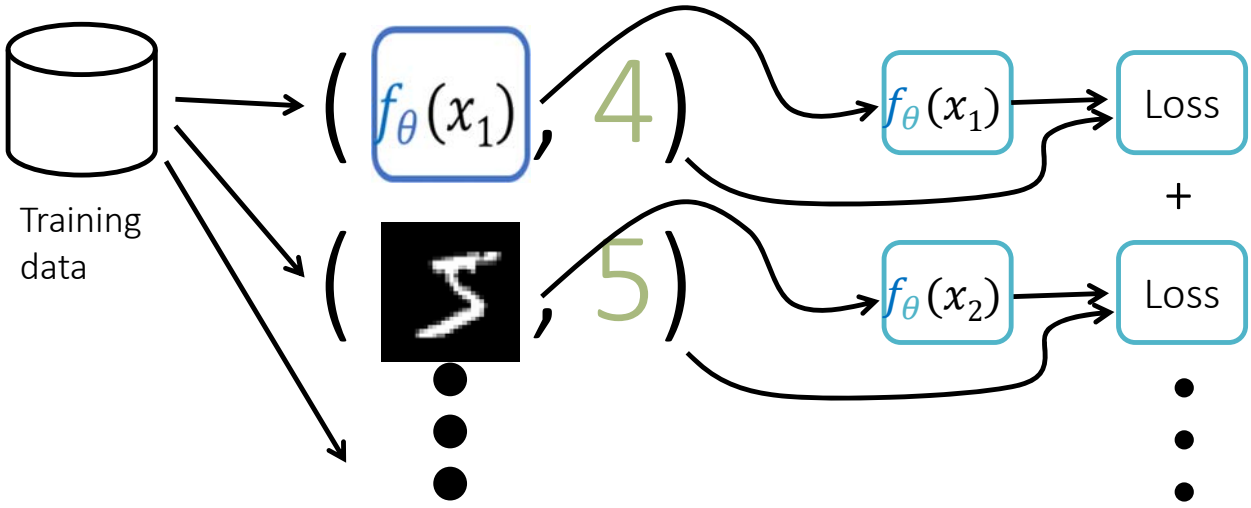
for a randomly chosen (x,y) from some distribution D.

- We don't know the distribution D.
- We only have access to (training) samples from D.

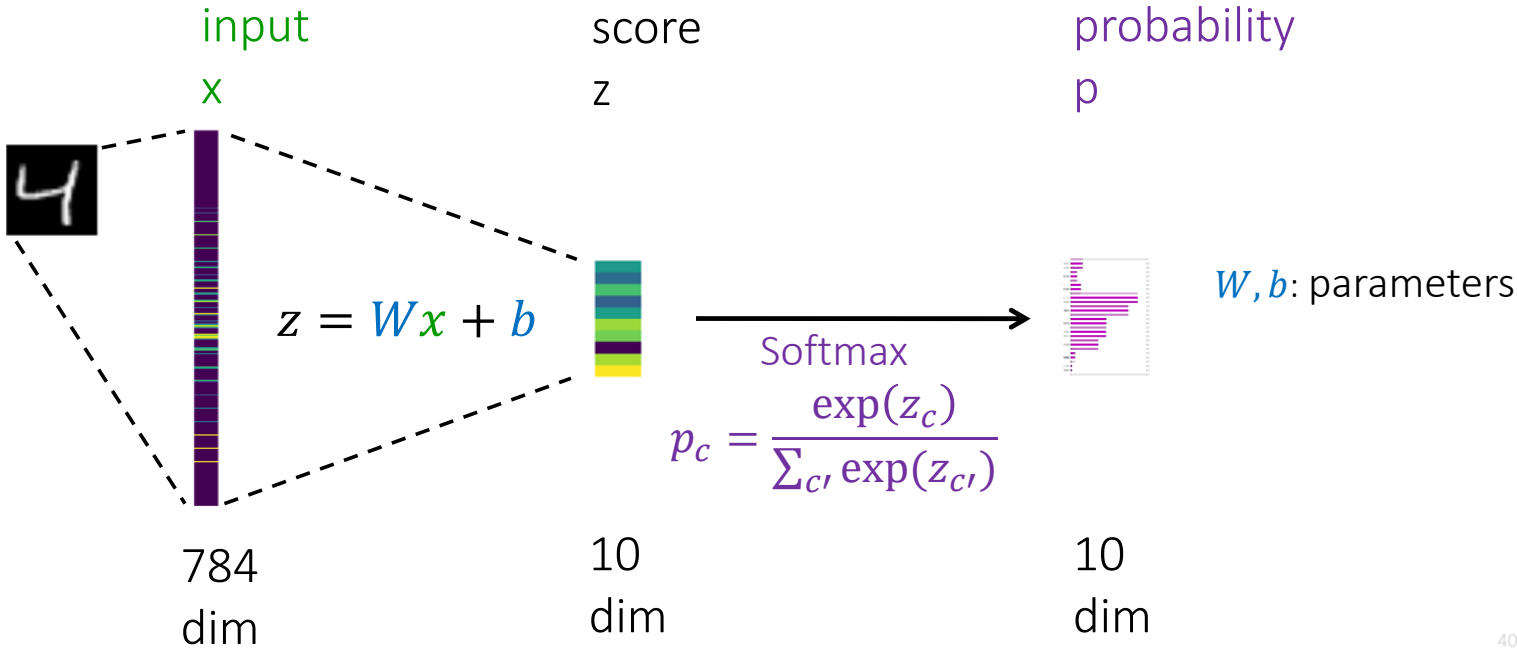
Training objective

Objective function: $L(\theta) = \frac{1}{n}\sum_{i=1}^{n} Loss(f_\theta(x_i), y_i)$

Approximate the unknown distribution *D* with the training data average

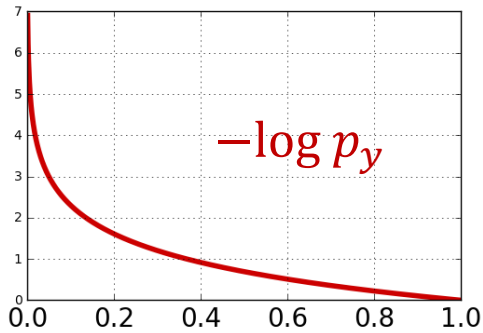


Mapping from input to prediction



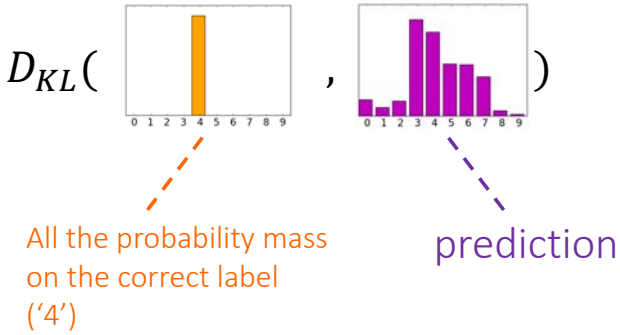
Cross-entropy loss

- Interpretation 1: you pay penalty



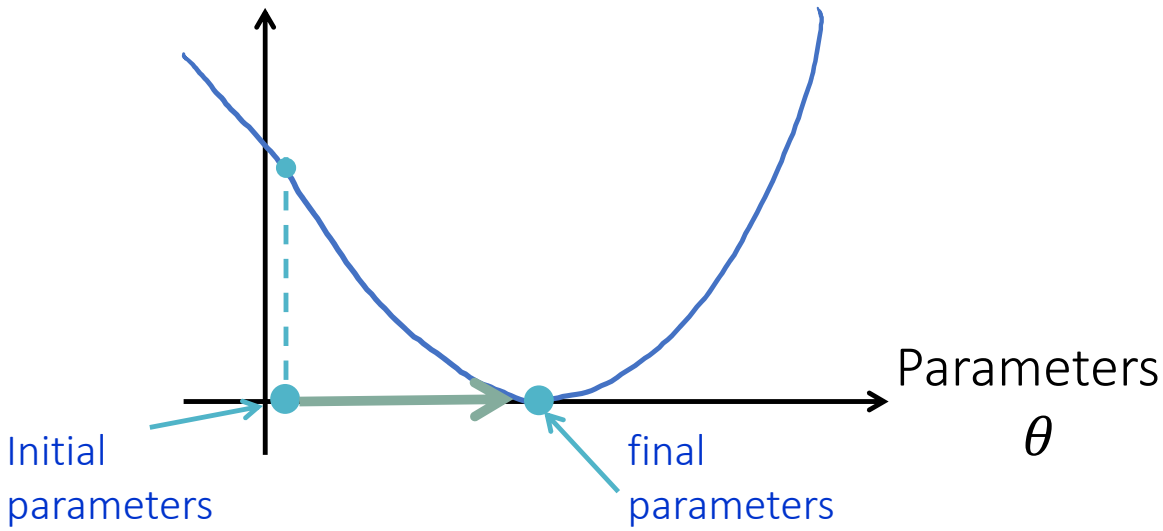
where y is the correct label.

- Interpretation 2: Kullback-Leibler divergence



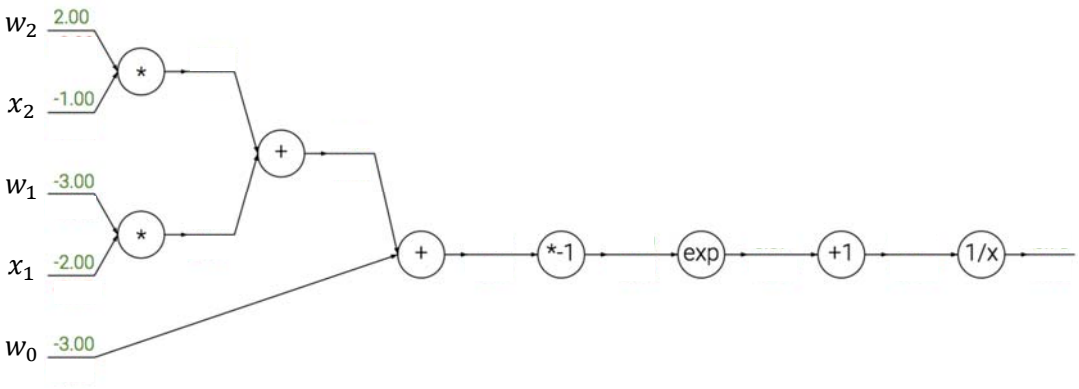
Landscape of training objective

- Objective function: $L(\theta) = \frac{1}{n}\sum_{i=1}^{n} Loss(f_\theta(x_i), y_i)$



A detailed example

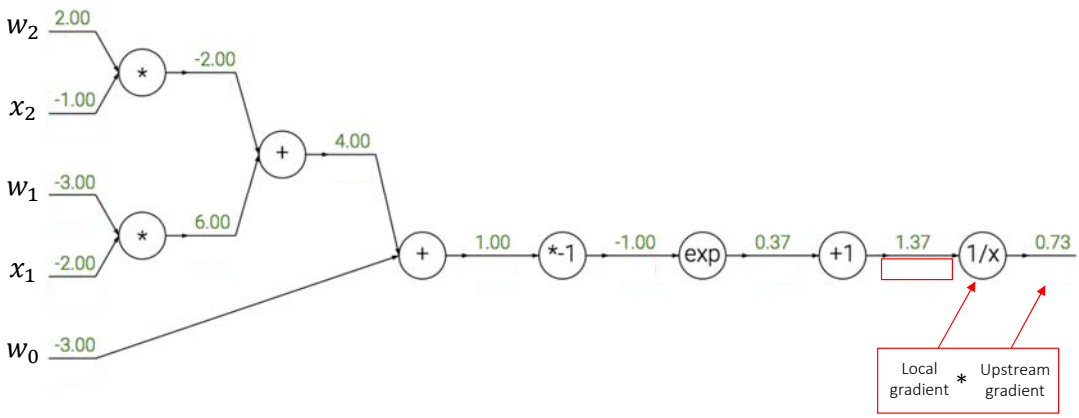
$$f(x) = \frac{1}{1 + \exp[-(w_0 + w_1 x_1 + w_2 x_2)]}$$



Source: Stanford 231n

A detailed example

$$f(x) = \frac{1}{1 + \exp[-(w_0 + w_1 x_1 + w_2 x_2)]}$$



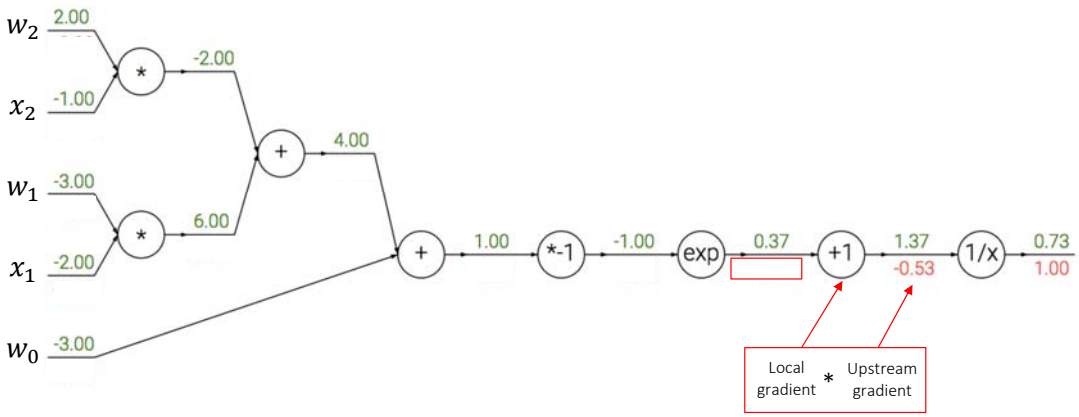
$$(1/x)' = -1/x^2$$

$$-\frac{1}{1.37^2} * 1 = -0.53$$

Source: Stanford 231n

A detailed example

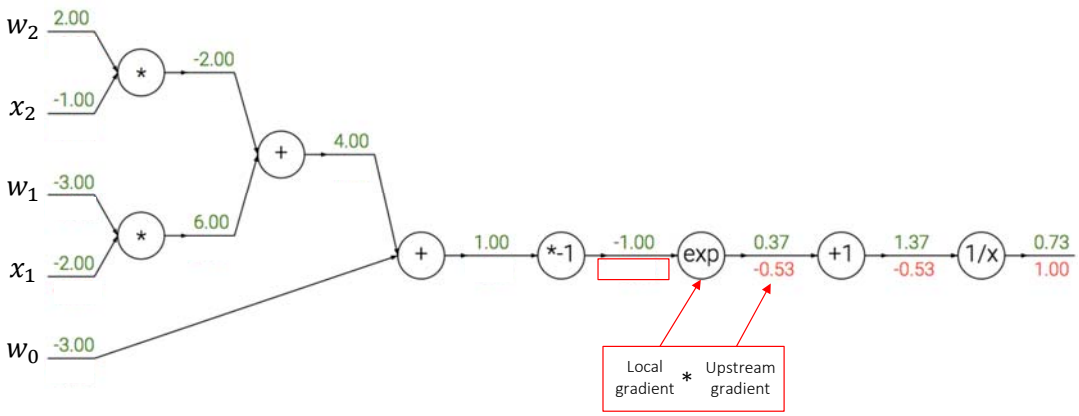
$$f(x) = \frac{1}{1 + \exp[-(w_0 + w_1 x_1 + w_2 x_2)]}$$



Source: Stanford 231n

A detailed example

$$f(x) = \frac{1}{1 + \exp[-(w_0 + w_1 x_1 + w_2 x_2)]}$$

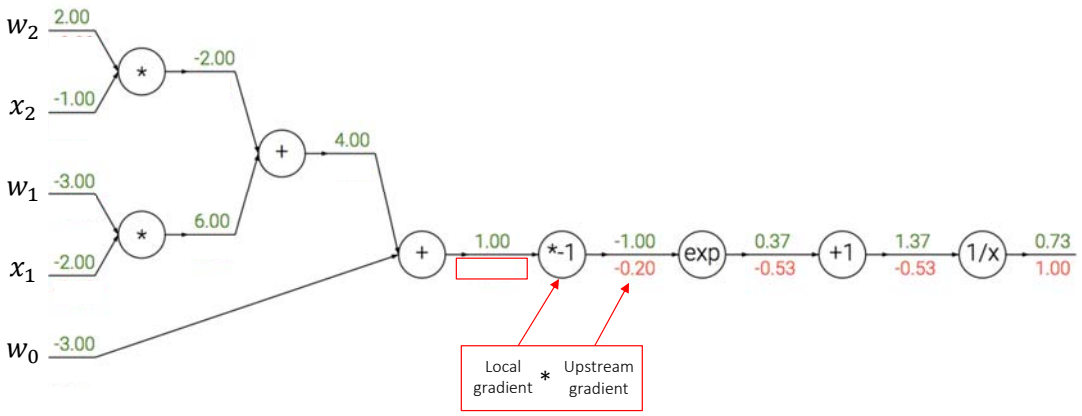


$$\exp(-1) * (-0.53) = -0.20$$

Source: Stanford 231n

A detailed example

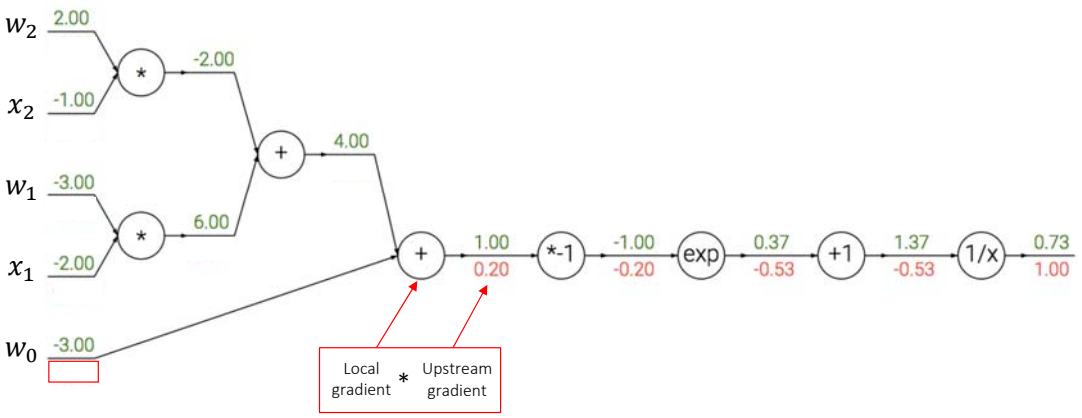
$$f(x) = \frac{1}{1 + \exp[-(w_0 + w_1 x_1 + w_2 x_2)]}$$



Source: Stanford 231n

A detailed example

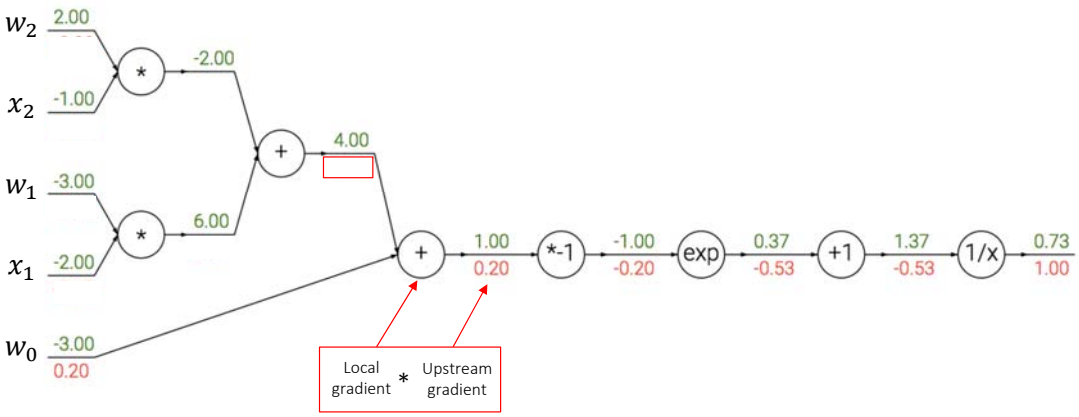
$$f(x) = \frac{1}{1 + \exp[-(w_0 + w_1 x_1 + w_2 x_2)]}$$



Source: Stanford 231n

A detailed example

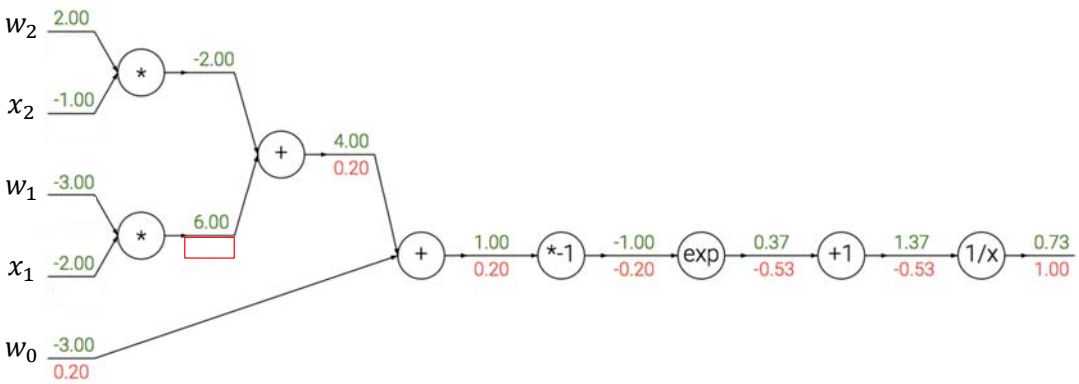
$$f(x) = \frac{1}{1 + \exp[-(w_0 + w_1 x_1 + w_2 x_2)]}$$



Source: Stanford 231n

A detailed example

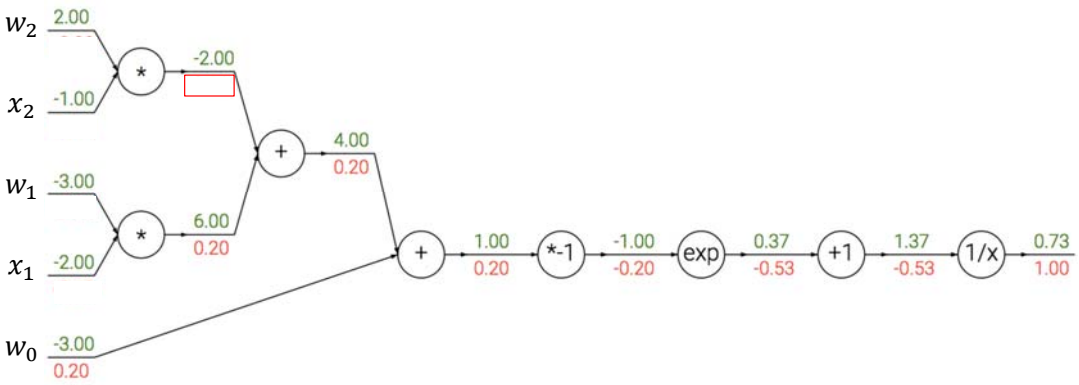
$$f(x) = \frac{1}{1 + \exp[-(w_0 + w_1 x_1 + w_2 x_2)]}$$



Source: Stanford 231n

A detailed example

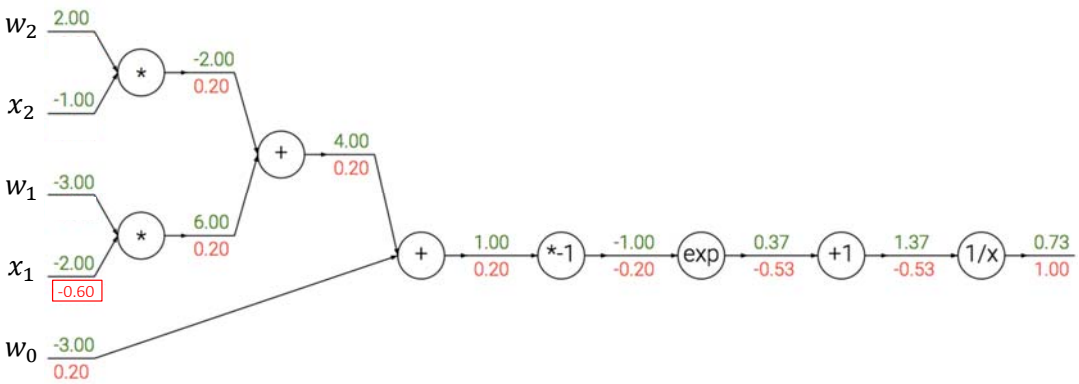
$$f(x) = \frac{1}{1 + \exp[-(w_0 + w_1 x_1 + w_2 x_2)]}$$



Source: Stanford 231n

A detailed example

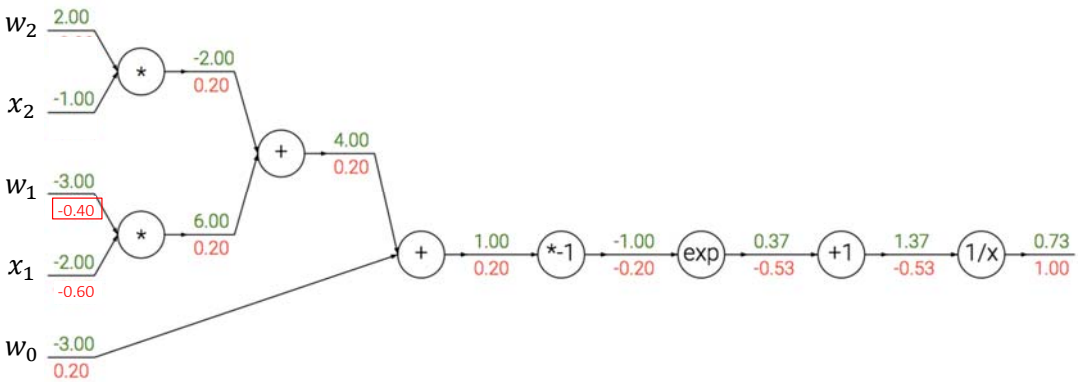
$$f(x) = \frac{1}{1 + \exp[-(w_0 + w_1 x_1 + w_2 x_2)]}$$



Source: Stanford 231n

A detailed example

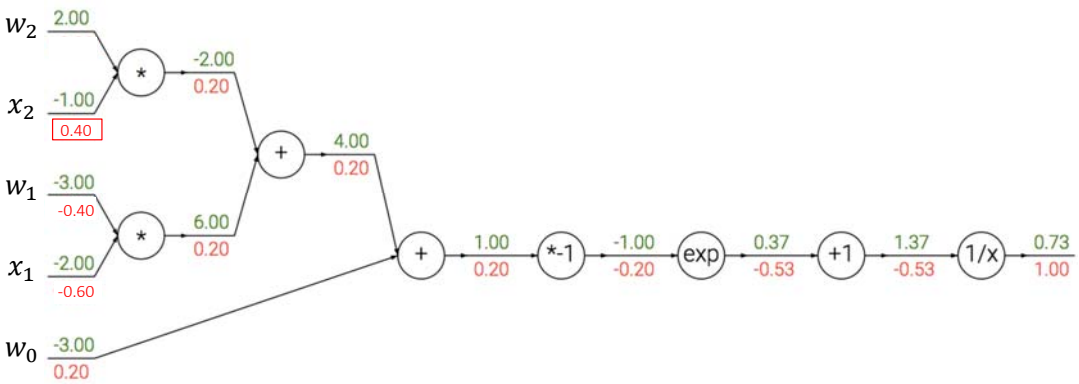
$$f(x) = \frac{1}{1 + \exp[-(w_0 + w_1 x_1 + w_2 x_2)]}$$



Source: Stanford 231n

A detailed example

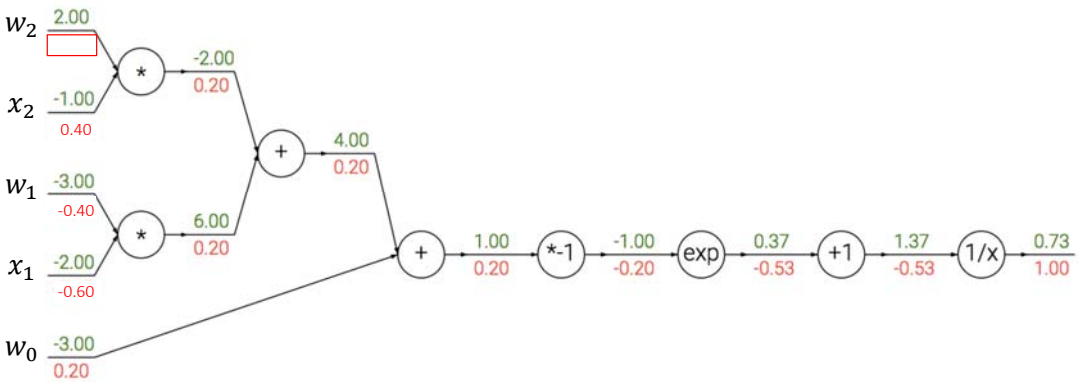
$$f(x) = \frac{1}{1 + \exp[-(w_0 + w_1 x_1 + w_2 x_2)]}$$



Source: Stanford 231n

A detailed example

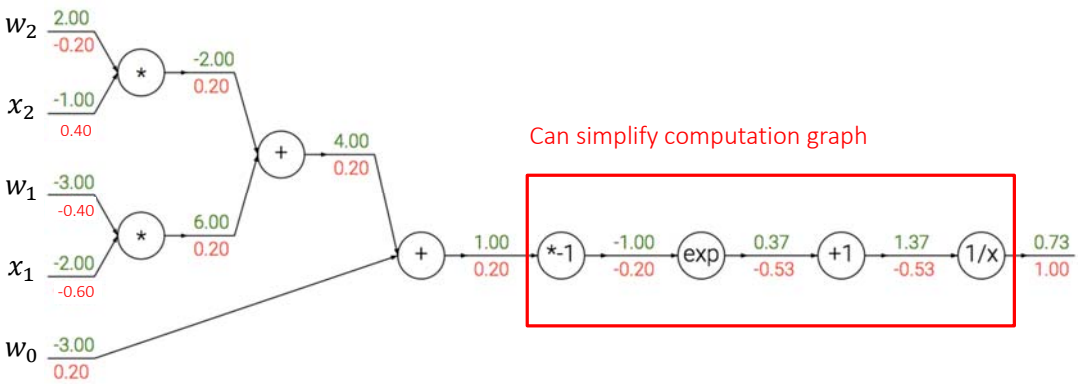
$$f(x) = \frac{1}{1 + \exp[-(w_0 + w_1 x_1 + w_2 x_2)]}$$



Source: Stanford 231n

A detailed example

$$f(z) = \frac{1}{1 + \exp[-(w_0 + w_1 x_1 + w_2 x_2)]}$$



Can simplify computation graph

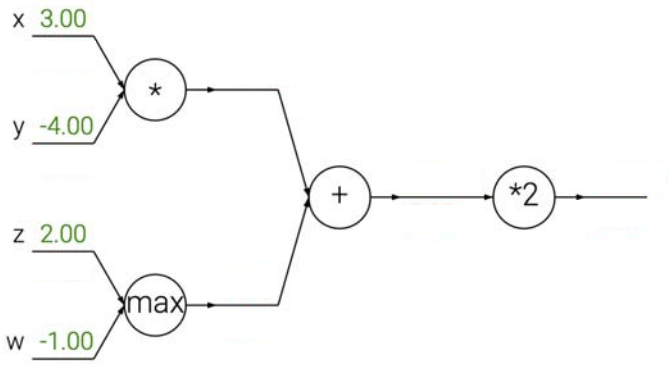
Sigmoid gate $\sigma(x)$

$$\sigma'(x) = \sigma(x)\big(1 - \sigma(x)\big)$$

$$\sigma(1)\big(1 - \sigma(1)\big) = 0.73 * (1 - 0.73) = 0.20$$

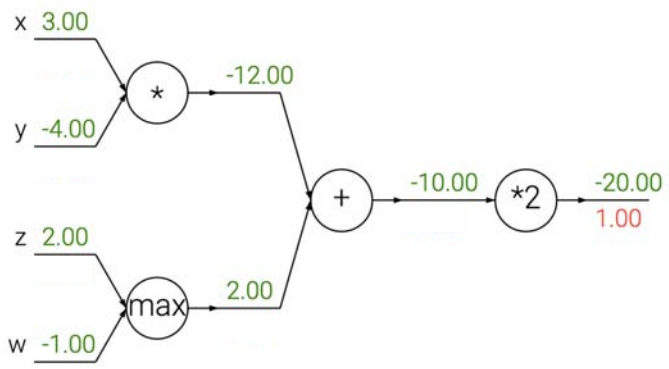
Source: Stanford 231n

Patterns in gradient flow



Source: Stanford 231n

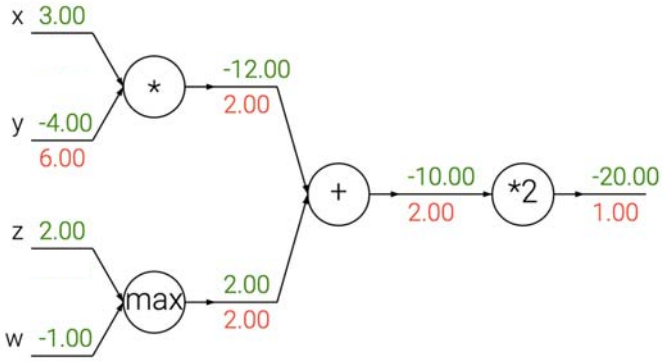
Patterns in gradient flow



Add gate: “gradient distributor”

Source: Stanford 231n

Patterns in gradient flow

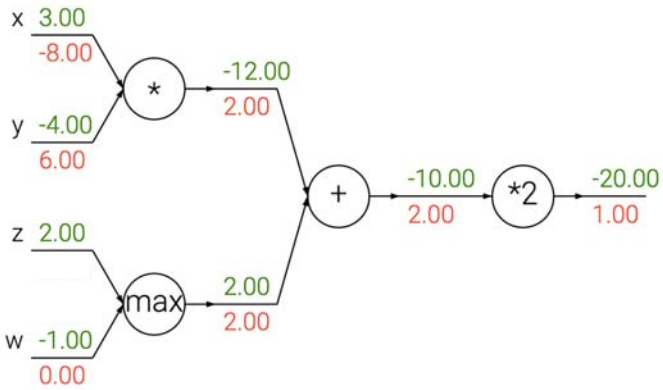


Add gate: “gradient distributor”

Multiply gate: “gradient switcher”

Source: Stanford 231n

Patterns in gradient flow



Add gate: “gradient distributor”

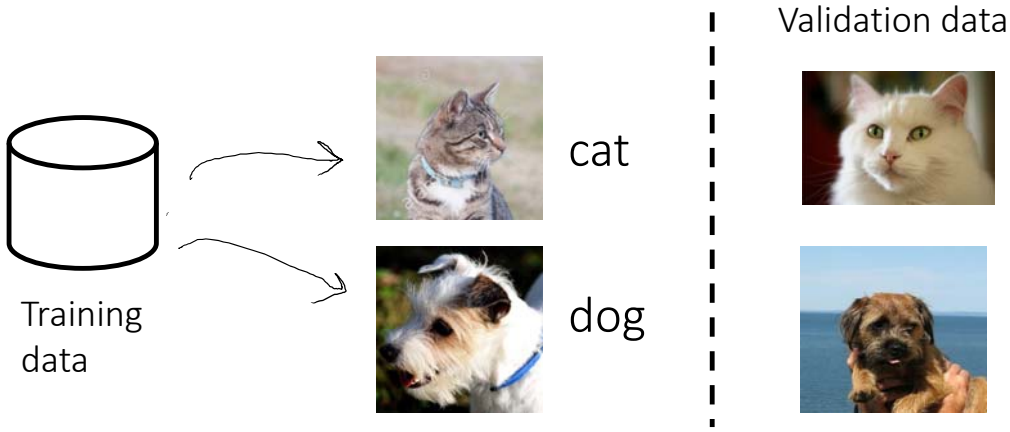
Multiply gate: “gradient switcher”

Max gate: “gradient router”

Source: Stanford 231n

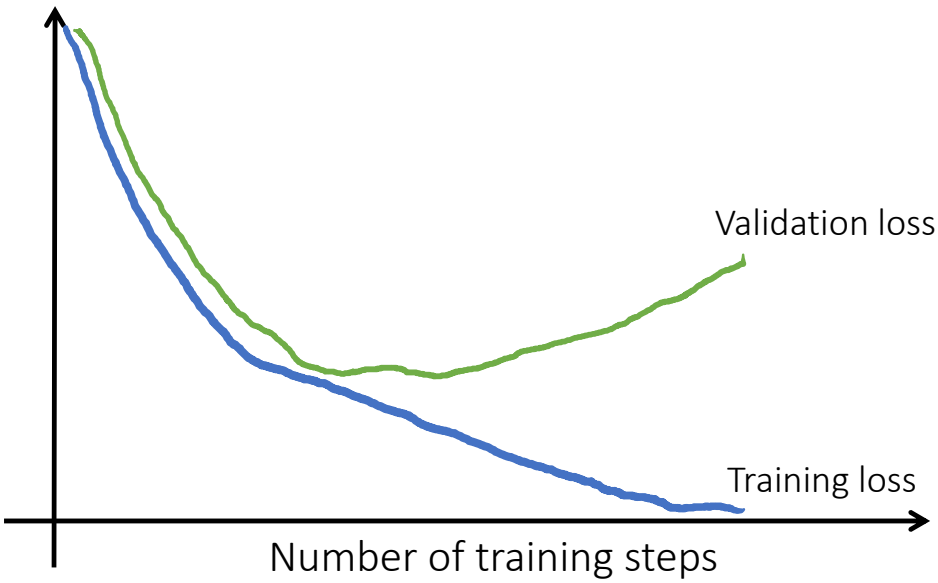
Overfitting – what is signal vs noise?

- Imagine:



- Powerful models are more likely to overfit
- We need validation data: leave out some portion of the training data to validate the generalizability of the model

Typical learning curve



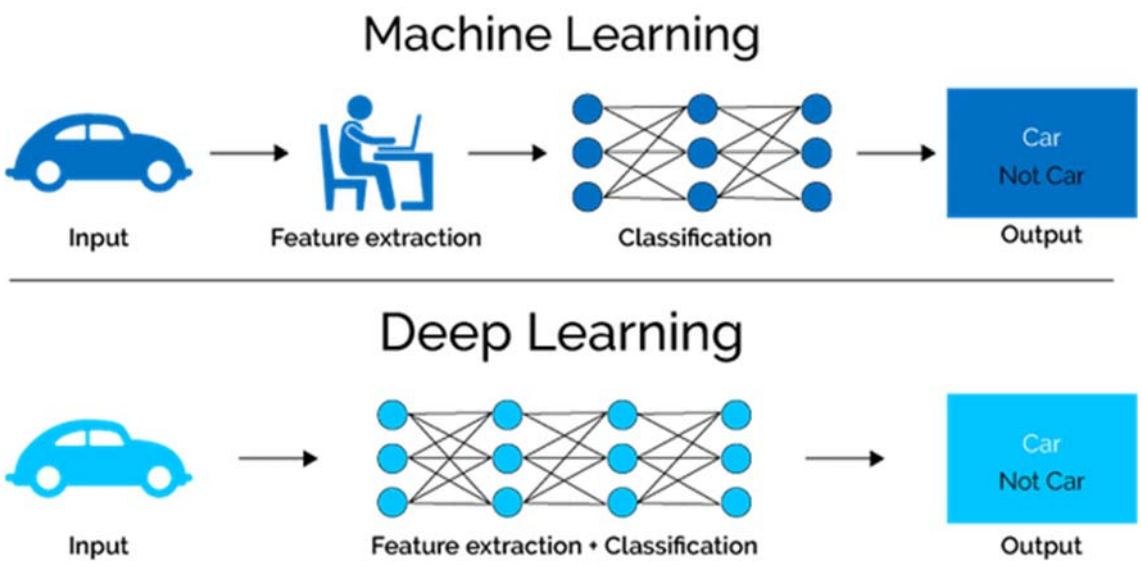
Summary so far

- A machine learning problem can be specified by:
  - *Task*: What's the input? What's the output?
  - *Model*: maps from the input to some numbers
  - *Loss function*: measures how the model is doing
  - *Training*: mini-batch SGD on the sum of *empirical* losses
  - *Validation*: Are we overfitting?

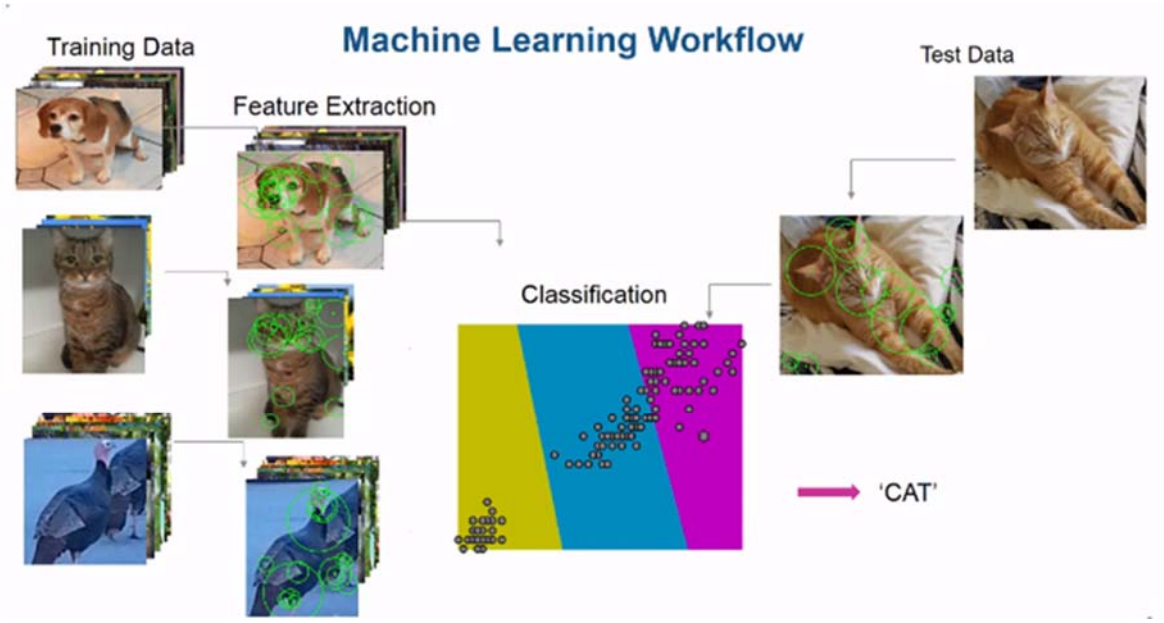
Techniques to reduce overfitting

- Reduce the number of parameters
  - Parameter sharing (convnets, recurrent neural nets)
- Weight decay (aka L2 regularization)
  - Penalizes the magnitude of the parameters $\sum_{j=1}^{d} w_j^2$
- Early stopping
  - Indirectly controls the magnitude of the parameters
- More recent techniques
  - Dropout, batch normalization

ML vs DL



Until Now…



Why Now ?

1952 Stochastic Gradient Descent

1958 Perceptron
- Learnable Weights

⋮

1986 Backpropagation
- Multi-Layer Perceptron

1995 Deep Convolutional NN
- Digit Recognition

⋮

Neural Networks date back decades, so why the resurgence?

1. Big Data

- Larger Datasets
- Easier Collection & Storage

IMAGENET

WIKIPEDIA The Free Encyclopedia

2. Hardware

- Graphics Processing Units (GPUs)
- Massively Parallelizable

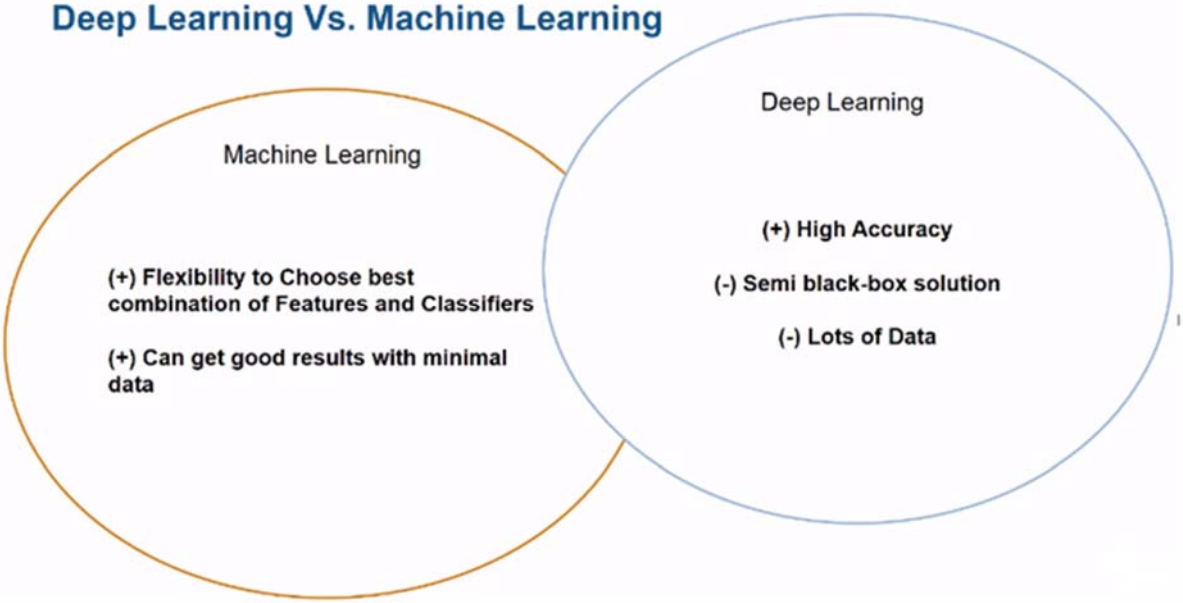
- Tensor Processing Units (TPUs)

3. Software

- Improved Techniques
- New Models
- Toolboxes

Chainer, mxnet, Caffe2, CNTK, TensorFlow, Keras, GLUON, PYTORCH, theano

Deep Learning Vs. Machine Learning



Deep Learning + Machine Learning Combined

Feature Learning

Deep Learning

Deep Learning AND Machine Learning

Combination Approach:

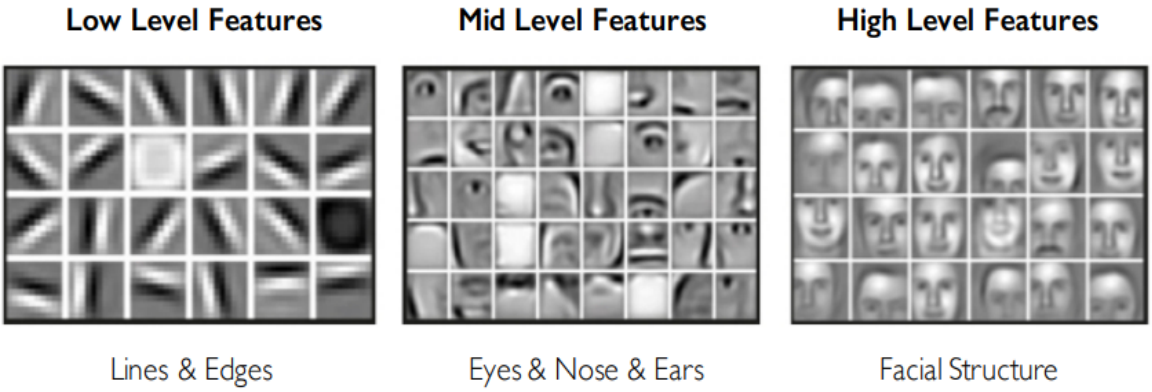
Deep Learning as a Feature Extractor

Machine Learning to create classifier

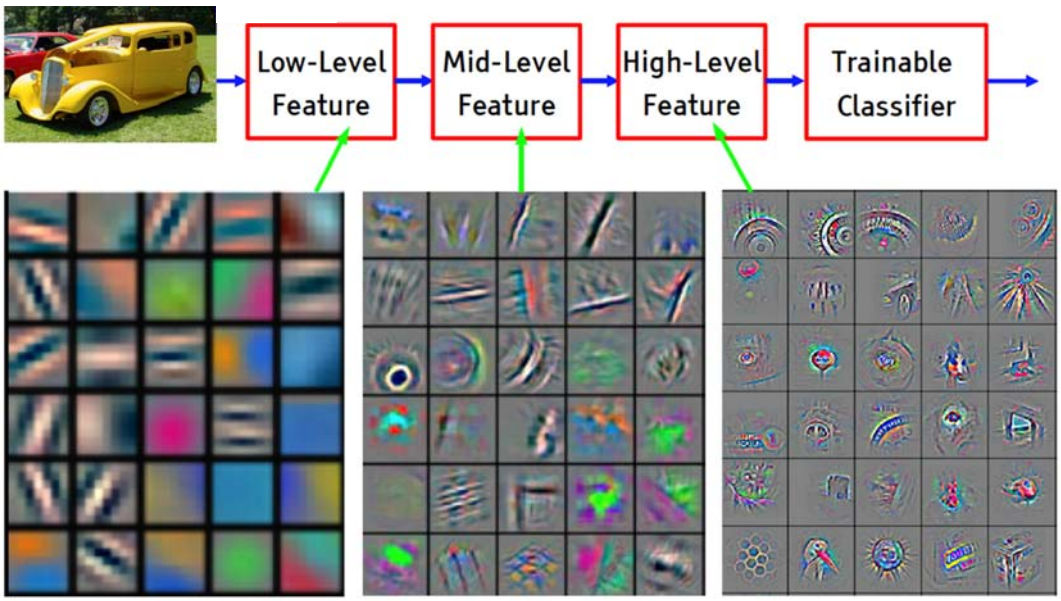
Remember: your results may vary
Do what's best for your problem and data!

Deep Learning = Learning Hierarchical Representations

- Hand engineered feature are time consuming, brittle and not scalable in practice.
- Can we learn the underlying features directly from data?



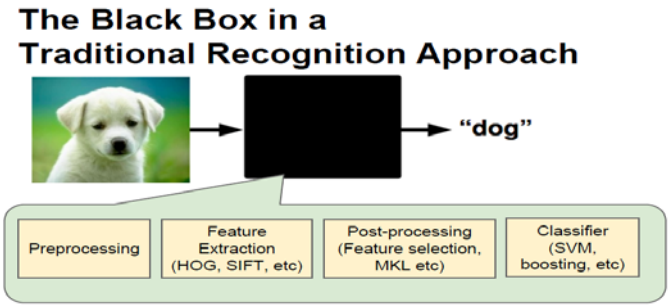
Deep Learning = Learning Hierarchical Representations

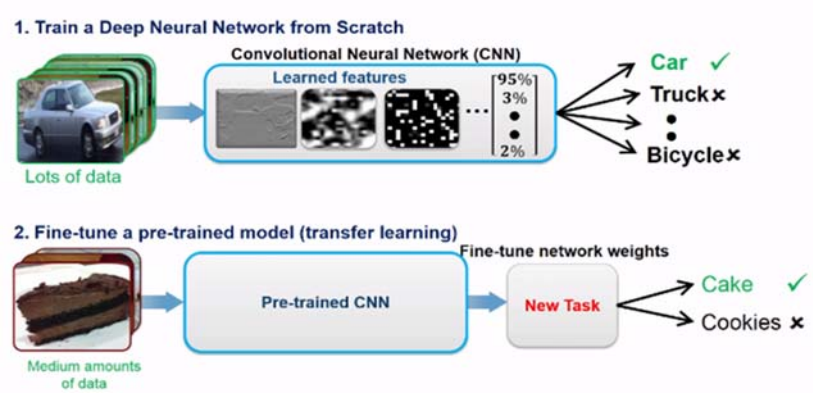


Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

Traditional Recognition Approach & DL

- A supervised deep learning method (Black box in traditional recognition approach)
- Convolutional Neural Networks of DL for recognition approach

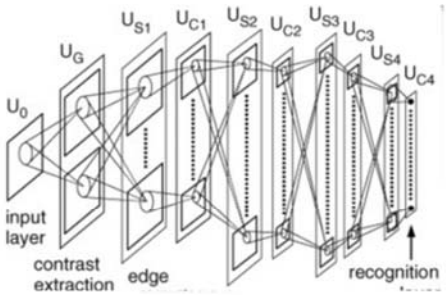


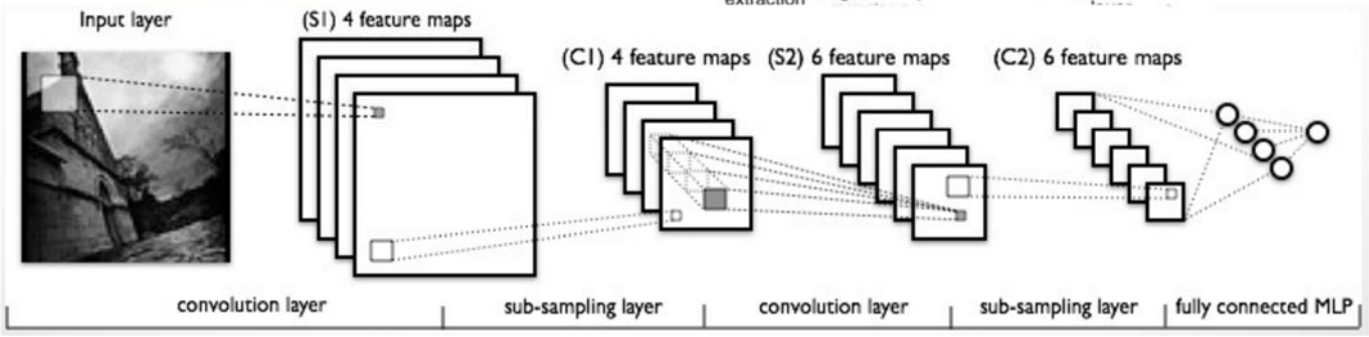


History of Deep Learning

- In 1971, Alexey Ivakhnenko (Father of DL) was a Soviet and Ukrainian mathematician most famous for developing the Group Method of Data Handling (GMDH) with eight layers of a deep network.
- In 1980, Kunihiko Fukushima, the Neocognitron was inspired by the model proposed by Hubel & Wiesel.
- In 1986, the term DL was introduced to the ML community by Rina Dechter.
- In 1989, Yann LeCun et al. applied the standard backpropagation algorithm to a deep neural network with the purpose of recognizing handwritten ZIP codes on mail. The Convolutional Neural Network was also inspired by this line concepts

History of Deep Learning



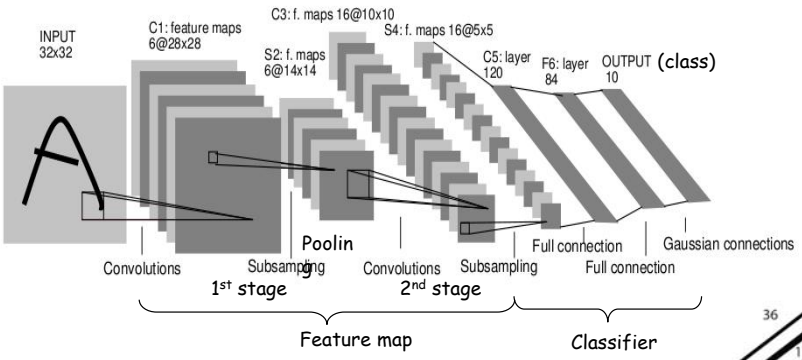


Types of Deep Learning

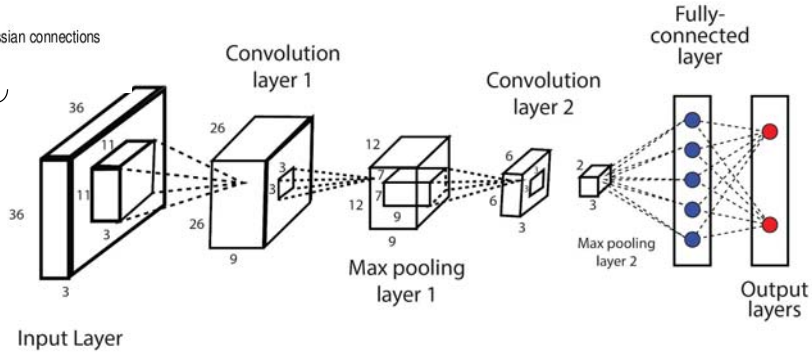
- Unsupervised learning
  - Deep Belief Networks (DBN), which are based on Restricted Boltzmann Machines (RBM)
  - Deep Autoencoders, which are based on Autoencoders.
- Supervised learning
  - Convolutional Neural Networks (CNNs), which are suitable for computer vision
  - Recurrent Neural Networks (RNNs), which are suitable for sequence data
  - Transformer, which are suitable for sequence data

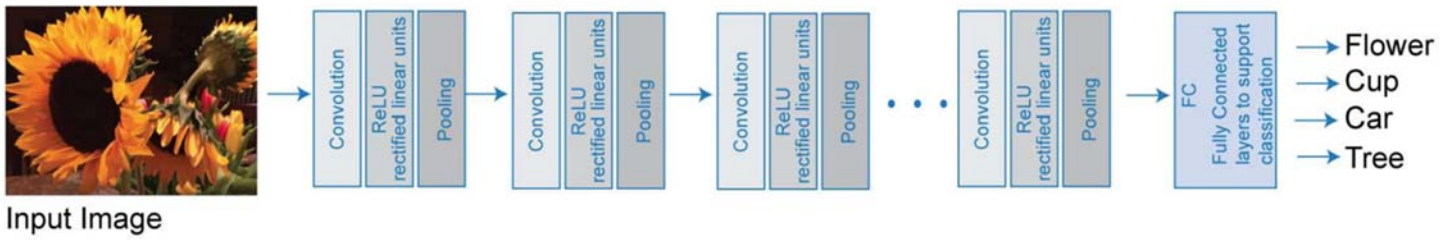
Start DL with Convolutional Neural Networks (CNN)

- LeNet-5 proposed by Lecun (1998).



- CNN
  - Convolution layer
  - Pooling layer
  - Fully connected layer



Why DL?

- Good performance
- End-to-end approach (representation learning)
- Challenges
  - Speed and accuracy in open-ended real-life problems
  - The whole ecosystem is still developing